Project Technical Documentation

MY EYES – VISION BEYOND SIGHT

Voice-First Assistive Artificial Intelligence & Passwordless Biometric Authentication Platform for Visually Impaired Individuals

---

Table of Contents

1. [Abstract](#1-abstract)

2. [Introduction & Project Overview](#2-introduction--project-overview)

3. [Hardware and Software Specification](#3-hardware-and-software-specification)

• 3.1 Hardware Requirements

• 3.2 Software & Framework Stack

4. [Module Description](#4-module-description)

• 4.1 Passwordless Biometric Authentication & Account Directory Module

• 4.2 Voice-First Companion & Command Router Module

• 4.3 Indian Currency Detection Engine

• 4.4 Real-Time Object & Scene Understanding Engine (YOLOv8)

• 4.5 Optical Character Recognition (OCR) & Product/Medicine Reader Engine

• 4.6 Smart GPS Navigation & Saved Places Module

• 4.7 Emergency SOS Alerting & Location Broadcasting System

• 4.8 Voice-Guided UPI Payment & QR Code Reader Module

• 4.9 AI Help Conversational Engine (Google Gemini 3.8 Flash)

5. [Data Flow Diagram (DFD)](#5-data-flow-diagram-dfd)

• 5.1 Level 0 Context DFD

• 5.2 Level 1 Detailed Subsystem DFD

6. [System Flow Diagram](#6-system-flow-diagram)

• 6.1 End-to-End Execution Flowchart

• 6.2 Voice & Vision Request State Lifecycle

7. [Entity-Relationship (ER) Diagram](#7-entity-relationship-er-diagram)

• 7.1 Database Entity Relationship Diagram

• 7.2 Relational Entity Matrix

8. [Database Design](#8-database-design)

• 8.1 Schema Architecture & Table Structures

• 8.2 Alembic Migrations & Indexing Strategy

9. [UI/UX Design & Accessibility Architecture](#9-uiux-design--accessibility-architecture)

• 9.1 Accessibility Design Principles

• 9.2 Wireframe Layouts & Voice Screen State Navigation

10. [API Specification & Route Architecture](#10-api-specification--route-architecture)

11. [Conclusion & Future Enhancements](#11-conclusion--future-enhancements)

12. [References](#12-references)

---

1. Abstract

Visually impaired individuals face significant daily challenges when identifying currency notes, reading labels on medicines and packaged goods, recognizing surrounding objects, navigating public spaces, and making digital payments. Existing assistive applications often suffer from fragmented functionality, forcing users to switch between multiple specialized apps, which increases cognitive load and degrades accessibility.

My Eyes – Vision Beyond Sight is a comprehensive, voice-first, AI-powered mobile and backend system engineered to provide complete visual assistance through a single unified interface. Operable via Google Assistant launch hooks and natural language voice commands, *My Eyes* integrates state-of-the-art computer vision models (YOLOv8, EasyOCR, OpenCV), cloud spatial intelligence APIs (Geoapify), generative AI assistance (Google Gemini 3.8 Flash), and passwordless biometric security (Android BiometricPrompt + SHA-256 hashed secret keys).

The platform delivers real-time voice feedback using custom voice companion profiles ("Alex" and "Maya"). Built on a high-performance **FastAPI** Python backend, a cross-platform **Flutter** frontend, and a **PostgreSQL** relational database, *My Eyes* provides an affordable, secure, and intuitive ecosystem that restores independence and confidence to visually impaired users.

• **Keywords**: Currency Detection, Medicine Recognition, Object Detection, OCR Text Reading, Voice Assistance, Passwordless Biometric Auth, Computer Vision, FastAPI, Flutter, OpenCV, YOLOv8, EasyOCR, Google Gemini.

---

2. Introduction & Project Overview

2.1 Problem Statement

Over 253 million people worldwide live with visual impairment, including 36 million who are blind. In their daily lives, simple tasks like identifying currency notes, verifying medicine expiration dates, recognizing objects in a room, finding nearby essential amenities, or invoking emergency help present major hurdles. Most existing solutions either require manual touch interactions on screen buttons—which are difficult for visually impaired users—or split features across disconnected single-purpose applications.

2.2 Project Vision

My Eyes addresses these limitations by establishing a **Voice-First, Vision-Powered Mobile Ecosystem**. The application prioritizes hands-free interaction: users launch the app via voice, speak natural commands, receive immediate high-quality text-to-speech (TTS) spoken guidance, and interact with the physical world through real-time computer vision analysis.

---

3. Hardware and Software Specification

3.1 Hardware Requirements

Mobile Client (End-User Device)

- **Processor**: Octa-core 2.0 GHz or higher (ARM64 architecture).
- **RAM**: Minimum 4 GB (6 GB recommended for smooth real-time camera inference).
- **Camera**: High-resolution rear camera (12 MP or higher with autofocus and LED flash).
- **Biometric Sensors**: Fingerprint scanner (Touch ID/Capacitive/In-Display) or 3D Face ID hardware supporting Android BiometricPrompt / iOS LocalAuthentication API.
- **Connectivity**: 4G LTE / 5G / Wi-Fi, GPS sensor, Accelerometer, Magnetometer, Microphone, High-volume Speaker.
- **Storage**: Minimum 150 MB free internal storage.

Backend Server (Host Infrastructure)

- **CPU**: 4-Core x86_64 Processor (Intel Xeon / AMD EPYC or modern equivalent).
- **RAM**: 8 GB RAM minimum (16 GB recommended when hosting PyTorch and EasyOCR concurrently).
- **GPU (Optional)**: NVIDIA GPU with CUDA support for accelerated batch OCR and YOLO inference.
- **Storage**: 20 GB SSD storage for operating system, dependencies, ML model weights (`yolov8n.pt`), and cached images.
- **Network**: Broadband connection with static IP / domain name, supporting TLS 1.3 HTTPS communication.

3.2 Software & Framework Stack

```
+-------------------------------------------------------------------------+
|                        MY EYES SYSTEM STACK                             |
+-------------------------------------------------------------------------+
|  FRONTEND MOBILE LAYER (Flutter / Dart)                                 |
|  - UI Framework: Flutter 3.x                                            |
|  - Voice STT/TTS: speech_to_text, flutter_tts                            |
|  - Security & Auth: local_auth, flutter_secure_storage                  |
|  - Camera & Motion: camera, geolocator, sensors_plus                    |
|  - Payments: Custom UPI QR Parser & Deep Linking                        |
+-------------------------------------------------------------------------+
                                  | HTTP / REST API (JSON)
                                  v
+-------------------------------------------------------------------------+
|  BACKEND API LAYER (FastAPI / Python 3.10+)                             |
|  - Web Framework: FastAPI (Uvicorn ASGI Server)                         |
|  - Security: PyJWT (JWT Tokens), PassLib (Bcrypt secret hashing)        |
|  - Database ORM: SQLAlchemy, Alembic (Database Migrations)              |
|  - AI & Vision Engines:                                                 |
|     * Object & Scene: Ultralytics YOLOv8 (yolov8n.pt)                   |
|     * Text OCR & Currency: EasyOCR, OpenCV (cv2)                        |
|     * Conversational AI: Google Gemini API (google-genai SDK)           |
|     * Smart Navigation: Geoapify REST Services (Geocoding & Places)     |
+-------------------------------------------------------------------------+
                                  |
                                  v
+-------------------------------------------------------------------------+
|  DATABASE STORAGE LAYER (PostgreSQL / SQLite)                           |
|  - Relational Database: PostgreSQL 14+ / SQLite (Dev)                   |
+-------------------------------------------------------------------------+
```

---

4. Module Description

4.1 Passwordless Biometric Authentication & Account Directory Module

- **Purpose**: Eliminates text fields and password inputs for blind users.
- **Device Biometric Verification**: Uses Android `BiometricPrompt` (`local_auth` package). Fingerprint data stays securely inside the operating system hardware enclave. The app never accesses raw biometric data.
- **Device Secret Hashing**: Upon initial account creation/linking, the mobile app generates a cryptographically random device secret and stores it in Android `EncryptedSharedPreferences`. The backend computes a SHA-256 hash of this secret (`DeviceCredential` model) and associates it with the user ID.
- **Google Account Directory Selector**: Uses Android `AccountManager.newChooseAccountIntent` filtered to `com.google`. It displays the Google accounts existing on the device, enabling single-touch selection without showing complex sign-in forms or triggering OAuth web popups.

4.2 Voice-First Companion & Command Router Module

- **Purpose**: Acts as the master orchestrator for hands-free app operation.
- **Natural Language Command Routing**: Captures user speech via `SpeechToText`, normalizes input, and routes commands using pattern matching and keyword extraction (`VoiceCommandRouter`).

- **Companion Voice Personalities**: Users can select between two companion voices: **Alex** (Male) or **Maya** (Female). Voice preferences are persisted in the backend database (`users.companion`) and synchronized with the frontend TTS engine.

4.3 Indian Currency Detection Engine

- **Purpose**: Accurately identifies Indian Rupee (INR) notes without relying on cloud vision fees.
- **Computer Vision Pipeline**: Combines EasyOCR text extraction with image enhancement (contrast adjustment, scaling, sharpening via OpenCV).
- **Multi-Tier Evidence Verification**: Evaluates note evidence across three strict rules to prevent misidentifying note serial numbers or price tags:

1. **Tier 1 (Printed Amount)**: Unambiguous currency text (■`100`, `RS 500`, `RUPEES 200`).
2. **Tier 2 (Word Denominations)**: Spelled-out numbers (`FIVE HUNDRED`, `ONE HUNDRED`) corroborated by currency markers (`RUPEES`).
3. **Tier 3 (Printed Numerals)**: Standalone numbers (`10`, `20`, `50`, `100`, `200`, `500`) requiring note markers (`RESERVE BANK OF INDIA`, `RBI`, `BHARAT`, `GANDHI`).

- **Supported Denominations**: ■10, ■20, ■50, ■100, ■200, ■500.

4.4 Real-Time Object & Scene Understanding Engine (YOLOv8)

- **Purpose**: Identifies surrounding physical objects and describes indoor/outdoor environments.
- **YOLOv8 Neural Network**: Utilizes lightweight `yolov8n.pt` weights to perform high-speed object detection.
- **Spatial Relationship Synthesis**: Analyzes bounding box coordinates (`x_min`, `y_min`, `x_max`, `y_max`) to calculate spatial positioning (e.g., "chair on your left", "bottle directly in front of you").

4.5 Optical Character Recognition (OCR) & Product/Medicine Reader Engine

- **Purpose**: Reads printed documents, packaging labels, and medical prescriptions.
- **Text Extraction**: Runs EasyOCR to extract line-by-line text blocks.
- **Regex Information Extraction**: Parses critical product metadata including Expiration Date (`EXP: 12/2028`), Manufacturing Date (`MFG: 01/2026`), Batch Number, Dosage, and Usage Instructions.

4.6 Smart GPS Navigation & Saved Places Module

- **Purpose**: Provides voice-guided outdoor spatial orientation and turn-by-turn navigation.
- **Geoapify Spatial Proxy**: Interfaces with Geoapify REST APIs for reverse geocoding, location lookup, and routing.
- **Saved Locations**: Allows users to save named coordinates (e.g., "Home", "Work", "Doctor Clinic") stored in the `saved_places` database table.

4.7 Emergency SOS Alerting & Location Broadcasting System

- **Purpose**: Protects users during emergencies through automatic multi-channel dispatch.
- **Triggers**: Voice command ("SOS", "Help me"), UI button press, or device double-shake detected via accelerometer.

- **Dispatch Pipeline**: Fetches real-time GPS coordinates, formats Google Maps location links, dispatches SMS alerts to prioritized emergency contacts (`emergency_contacts`), and logs the event in `sos_events`.

4.8 Voice-Guided UPI Payment & QR Code Reader Module

- **Purpose**: Enables visually impaired users to pay safely via UPI QR codes.
- **QR Reader**: Captures camera frames and parses standard UPI payment URIs (`upi://pay?pa=recipient@upi&pn=MerchantName&am=150`).
- **Voice Amount Confirmation**: Reads out merchant name and payment amount, prompts for spoken user confirmation, and opens installed payment apps (PhonePe, Google Pay, Paytm) via deep linking.

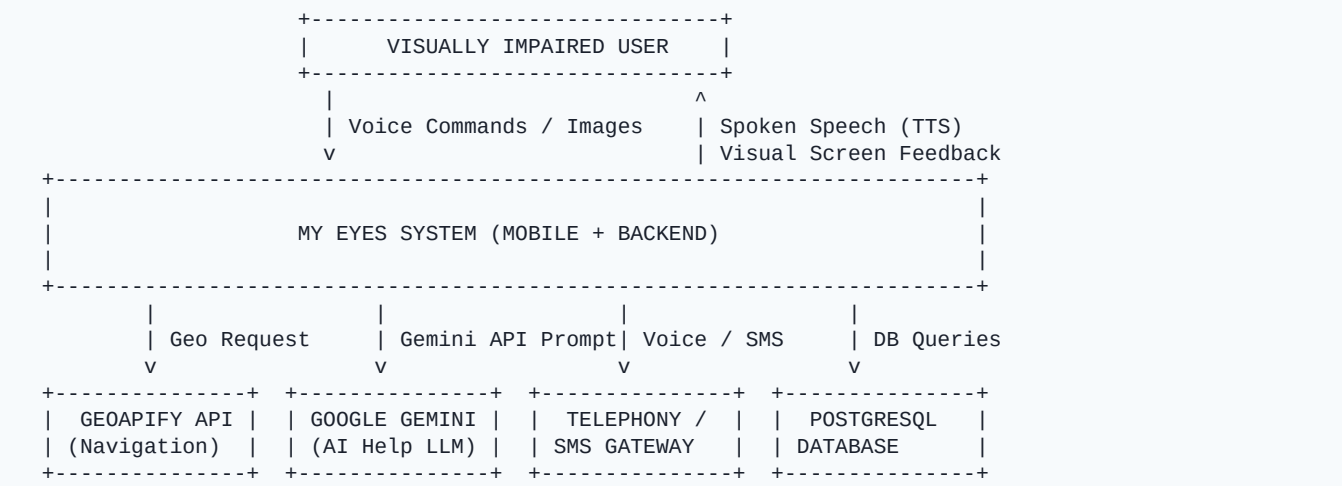
4.9 AI Help Conversational Engine (Google Gemini 3.8 Flash)

- **Purpose**: Answers general user questions, provides reasoning, and converses naturally.
- **Google Gemini Integration**: Uses the official `google-genai` Python SDK calling the `gemini-3.8-flash` model.
- **Turn Context Management**: Maintains rolling conversation history (`ai_conversations` and `ai_messages` tables), capping active context to 10 turns (`_MAX_CONTEXT_TURNS=10`) for minimal latency and spoken TTS friendliness.

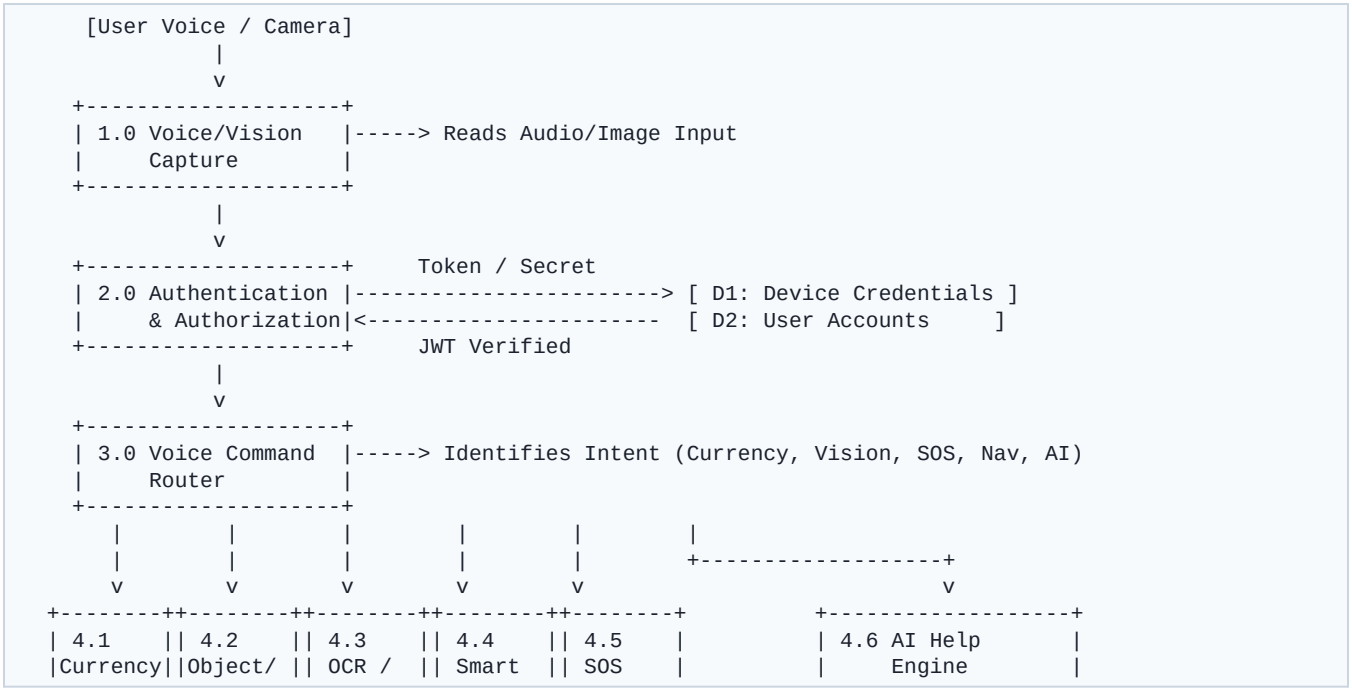
---

5. Data Flow Diagram (DFD)

5.1 Level 0 Context DFD



5.2 Level 1 Detailed Subsystem DFD



6. System Flow Diagram

6.1 Master Execution Flowchart

```
flowchart TD
    Start([User Opens My Eyes App]) --> CheckAuth{Is Device Authenticated?}

    CheckAuth -- No --> GoogleAccount[Select Account via Android AccountManager]
    GoogleAccount --> BioPrompt[Trigger Android Fingerprint / Face BiometricPrompt]
    BioPrompt -- Success --> GenSecret[Generate Device Secret & Register SHA256 Hash]
    BioPrompt -- Failure --> RetryBio[Speak Error & Prompt Retry]
    RetryBio --> BioPrompt
    GenSecret --> IssueJWT[Backend Issues JWT Access Token]

    CheckAuth -- Yes --> IssueJWT
    IssueJWT --> ActiveState[App enters Voice Listening State]

    ActiveState --> CaptureSpeech[Capture Voice Command or Camera Action]
    CaptureSpeech --> MatchIntent{Identify Intent}

    MatchIntent -- "Currency" --> CaptureImg1[Capture Note Image]
    CaptureImg1 --> OCRProc1[Run EasyOCR + Note Marker Analysis]
    OCRProc1 --> DenomCheck{Denomination Found?}
    DenomCheck -- Yes --> SpeakRes1[Speak: 'This is a 100 Rupee Note']
    DenomCheck -- No --> SpeakRes1Err[Speak: 'Currency not recognized clearly']

    MatchIntent -- "Object / Scene" --> CaptureImg2[Capture Camera Frame]
    CaptureImg2 --> YOLOProc[Run YOLOv8 Inference & Spatial Mapping]
    YOLOProc --> SpeakRes2[Speak Identified Objects & Positions]
```

```

    MatchIntent -- "OCR / Medicine" --> CaptureImg3[Capture Text/Label Image]
    CaptureImg3 --> OCRProc2[Run EasyOCR & Pattern Extraction]
    OCRProc2 --> SpeakRes3[Speak Text / Expiry Date / Dosage]

    MatchIntent -- "Navigation" --> GeoReq[Get GPS Coords & Call Geoapify Proxy]
    GeoReq --> SpeakRes4[Speak Turn-by-Turn Navigation / Nearby Places]

    MatchIntent -- "SOS" --> GetGPS[Acquire High-Accuracy GPS Location]
    GetGPS --> SendSMS[Broadcast SMS & Call Primary Emergency Contacts]
    SendSMS --> SpeakRes5[Speak: 'SOS Sent to Emergency Contacts']

    MatchIntent -- "AI Help" --> SendGemini[Forward Query to Gemini 3.8 Flash]
    SendGemini --> SpeakRes6[Speak Assistant Response]

    SpeakRes1 --> LogHistory[Save Action in DB scan_history]
    SpeakRes1Err --> LogHistory
    SpeakRes2 --> LogHistory
    SpeakRes3 --> LogHistory
    SpeakRes4 --> LogHistory
    SpeakRes5 --> LogHistory
    SpeakRes6 --> LogHistory

    LogHistory --> ActiveState
```

7. Entity-Relationship (ER) Diagram

7.1 Database Entity Relationship Diagram

```
erDiagram
    USERS ||--o{ DEVICE_CREDENTIALS : "registers"
    USERS ||--o{ EMERGENCY_CONTACTS : "defines"
    USERS ||--o{ SOS_EVENTS : "triggers"
    USERS ||--o{ SCAN_HISTORY : "performs"
    USERS ||--o{ SAVED_PLACES : "saves"
    USERS ||--o{ AI_CONVERSATIONS : "owns"
    AI_CONVERSATIONS ||--o{ AI_MESSAGES : "contains"

    USERS {
        int id PK
        string full_name
        string email UK
        string password "Nullable"
        string phone_number
        string companion "alex / maya"
        datetime created_at
    }

    DEVICE_CREDENTIALS {
        int id PK
        int user_id FK
        string device_id UK
        string secret_hash
        datetime created_at
```

```
        datetime last_used_at
    }

    EMERGENCY_CONTACTS {
        int id PK
        int user_id FK
        string name
        string phone_number
        string relation
        int priority "1=Primary, 2=Secondary"
        datetime created_at
        datetime updated_at
    }

    SOS_EVENTS {
        int id PK
        int user_id FK
        datetime triggered_at
        float latitude
        float longitude
        string status "TRIGGERED/SENT/FAILED"
        int contact_count
    }

    SCAN_HISTORY {
```

```
        int id PK
        int user_id FK
        string scan_type
        string input_image
        text result
        datetime created_at
    }

    SAVED_PLACES {
        int id PK
        int user_id FK
        string name
        float latitude
        float longitude
        datetime created_at
        datetime updated_at
    }

    AI_CONVERSATIONS {
        int id PK
        int user_id FK
        datetime created_at
        datetime updated_at
    }
```

```
    AI_MESSAGES {
        int id PK
        int conversation_id FK
        string role "user / assistant"
        text message
        datetime created_at
    }
```

---

8. Database Design

8.1 Schema Architecture & Table Structures

1. Table: `users`

Stores registered user profile data and voice companion preference.

Column Name	Data Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Primary key identifier
`full_name`	VARCHAR(100)	NOT NULL	User's full name
`email`	VARCHAR(100)	UNIQUE, NOT NULL	Account email address
`password`	VARCHAR(255)	NULLABLE	Password hash (NULL for passwordless)
`phone_number`	VARCHAR(15)	NULLABLE	Contact phone number
`companion`	VARCHAR(20)	NULLABLE	Selected voice companion (`alex` / `maya`)
`created_at`	DateTime	DEFAULT UTC	Timestamp of account creation

2. Table: `device_credentials`

Stores device authentication secrets for passwordless login.

Column Name	Data Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Primary key identifier
`user_id`	Integer	FK -> `users.id` (CASCADE)	User associated with device
`device_id`	VARCHAR(64)	UNIQUE, NOT NULL	Unique hardware identifier
`secret_hash`	VARCHAR(255)	NOT NULL	SHA-256 hash of device-side secret
`created_at`	DateTime	DEFAULT UTC	Device enrollment timestamp
`last_used_at`	DateTime	DEFAULT UTC	Last successful login timestamp

3. Table: `emergency_contacts`

Contains prioritized emergency contacts for SOS dispatch.

Column Name	Data Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Primary key identifier
`user_id`	Integer	FK -> `users.id` (CASCADE)	Owner user ID
`name`	VARCHAR(100)	NOT NULL	Contact name
`phone_number`	VARCHAR(20)	NOT NULL	Phone number for SMS/Call
`relation`	VARCHAR(50)	NOT NULL	Relationship (e.g., Parent, Spouse)
`priority`	Integer	DEFAULT 2	Contact priority order (1 = Highest)
`created_at`	DateTime	DEFAULT UTC	Creation timestamp
`updated_at`	DateTime	ON UPDATE UTC	Modification timestamp

4. Table: `sos_events`

Audit log of all triggered emergency alerts.

Column Name	Data Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Event identifier
`user_id`	Integer	FK -> `users.id` (CASCADE)	User who triggered SOS
`triggered_at`	DateTime	DEFAULT UTC	SOS timestamp
`latitude`	Float	NULLABLE	GPS latitude coordinate
`longitude`	Float	NULLABLE	GPS longitude coordinate
`status`	VARCHAR(20)	DEFAULT 'TRIGGERED'	Alert status (`TRIGGERED`, `SENT`, `FAILED`)
`contact_count`	Integer	DEFAULT 0	Number of contacts notified

5. Table: `scan_history`

Logs all visual vision scans, currency detections, and OCR readings.

Column Name	Data Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Record identifier
`user_id`	Integer	FK -> `users.id` (CASCADE)	User ID
`scan_type`	VARCHAR(50)	NOT NULL	Feature type (`currency`, `ocr`, `object`, etc.)
`input_image`	VARCHAR(255)	NULLABLE	Stored image file path
`result`	Text	NOT NULL	Spoken text / JSON outcome
`created_at`	DateTime	DEFAULT UTC	Scan timestamp

6. Table: `saved_places`

Persists user's favorite locations for smart voice navigation.

Column Name	Data Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Place identifier
`user_id`	Integer	FK -> `users.id` (CASCADE)	User ID
`name`	VARCHAR(100)	NOT NULL	Custom place label (e.g., "Home")
`latitude`	Float	NOT NULL	Latitude coordinate
`longitude`	Float	NOT NULL	Longitude coordinate
`created_at`	DateTime	DEFAULT UTC	Creation timestamp
`updated_at`	DateTime	ON UPDATE UTC	Modification timestamp

7. Table: `ai_conversations` & `ai_messages`

Manages Gemini AI help chat context.

- `ai_conversations`: `id` (PK), `user_id` (FK -> `users.id`), `created_at`, `updated_at`.
- `ai_messages`: `id` (PK), `conversation_id` (FK -> `ai_conversations.id`), `role` (`user`/`assistant`), `message` (Text), `created_at`.

---

9. UI/UX Design & Accessibility Architecture

9.1 Accessibility Design Principles

1. **Voice-First Paradigm**: Every action can be executed via voice commands.
2. **High-Contrast Touch Regions**: Large color-coded screen buttons (e.g., bright Red for SOS, bright Blue for Mic, bright Green for Currency) designed for users with partial vision.

3. **Screen Reader Compatibility**: Full support for Android TalkBack and iOS VoiceOver with custom semantic labels on all controls.
4. **Haptic Audio Feedback**: Distinct vibration pattern pulses confirm button presses, voice listening states, and error alerts.

---

10. API Specification & Route Architecture

Method	Endpoint	Description	Auth Required
POST	`/auth/register`	Register new user account	No
POST	`/auth/login`	Passwordless biometric login	No
POST	`/auth/device-login`	Authenticate using device secret hash	No
GET	`/profile`	Get user profile & voice companion setting	Yes (JWT)
PUT	`/profile/companion`	Update companion voice (`alex` / `maya`)	Yes (JWT)
POST	`/currency/detect`	Upload image for currency denomination detection	Yes (JWT)
POST	`/ocr/read`	Upload image for text OCR extraction	Yes (JWT)
POST	`/product-reader/read`	Upload product label for expiry/details extraction	Yes (JWT)
POST	`/object-detection/detect`	Upload image for YOLOv8 object identification	Yes (JWT)
POST	`/scene/describe`	Upload image for spatial scene description	Yes (JWT)
GET	`/navigation/nearby`	Search nearby amenities (hospitals, ATMs, pharmacies)	Yes (JWT)
POST	`/navigation/directions`	Calculate turn-by-turn routing steps	Yes (JWT)
GET	`/emergency-contacts`	Fetch emergency contacts list	Yes (JWT)
POST	`/emergency-contacts`	Add new emergency contact	Yes (JWT)
POST	`/sos/trigger`	Broadcast SOS alert to emergency contacts	Yes (JWT)
POST	`/ai/ask`	Send question to Google Gemini 3.8 Flash AI Help	Yes (JWT)

---

11. Conclusion & Future Enhancements

11.1 Conclusion

My Eyes – Vision Beyond Sight successfully unifies multiple cutting-edge AI technologies into an intuitive, voice-first mobile platform. By combining real-time object detection (YOLOv8), multi-tier currency verification (EasyOCR), generative conversation (Google Gemini), smart navigation (Geoapify), emergency broadcasting, and passwordless biometric security, the application provides a dependable digital assistant that significantly enhances the independence of visually impaired individuals.

11.2 Future Enhancements

1. **Offline Vision Models**: Quantize YOLOv8 and EasyOCR models to run locally on mobile hardware via TensorFlow Lite / ONNX Runtime when internet access is unavailable.
2. **Smart Glasses Integration**: Extend screen streaming and voice control to wearable smart glasses (e.g., Ray-Ban Meta / custom ESP32 camera frames).
3. **Advanced Indoor Navigation**: Integrate Ultra-Wideband (UWB) and Bluetooth Low Energy (BLE) beacons for high-precision indoor room navigation.

---

12. References

1. Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv preprint arXiv:1804.02767.
2. EasyOCR Documentation & PyTorch Backend: https://github.com/JaidedAI/EasyOCR
3. FastAPI Framework Documentation: https://fastapi.tiangolo.com/
4. Flutter Framework & Accessibility Guidelines: https://docs.flutter.dev/development/accessibility-and-localization/accessibility
5. Google Gemini API Documentation: https://ai.google.dev/gemini-api/docs
6. Geoapify Spatial & Routing API: https://www.geoapify.com/docs
7. Android BiometricPrompt Architecture: https://developer.android.com/training/sign-in/biometric-auth